

MECHATRONIC SYSTEM INTEGRATION

(MCTA 3203)

SEMESTER 1, 2026/2027

EXPERIMENT 1

DIGITAL LOGIC SYSTEM

(MCTA 3203)

SECTION 1

GROUP 4

NO.	MATRIC NO.	NAME
1	2413381	HARITH NAUFAL BIN AHMAD SHUKRI
2	2418123	IMAN BIN SALMAN
3	2411349	LUQMAN HAKIMI BIN SUHAIZI
4	2418929	MUHAMAD HAZIQ BIN MOHD SHAHRIL

SUBMISSION DATE:

ABSTRACT

This experiment demonstrated digital output control by interfacing a common-anode seven-segment display and two pushbuttons with an Arduino Uno. Pins D2–D8 controlled the display segments using active-LOW logic, while pins D9 and D10 read the count-up and reset buttons using internal pull-up resistors and debounce delays. The key objective of this experiment is to have a digital counter that display numerical value and allow user to increase or reset the displayed number using two separate push buttons.

First, the circuit was built according to the required connections with the display and two pushbuttons connected to the Arduino. The Arduino program was then uploaded, and the counter was initialized at 0. The count-up button was pressed to increase the displayed number and finally, the reset button was tested to ensure the display returned to 0. During testing, the circuit operated reliably without issues: pressing the count-up button incremented the displayed number from 0 to 9, and the reset button cleared it back to 0.

The challenges that we overcame during the experiment included connecting the circuit correctly and ensuring that all components were properly connected to the Arduino. We solved these issues by checking the wiring connections carefully and correcting any mistakes. Potential future improvements include writing modular segment-control code, adding software debouncing, and expanding the design to multi-digit displays so it can count higher numbers more accurately and reliably.

TABLE OF CONTENTS

ABSTRACT....2
TABLE OF CONTENTS....3
INTRODUCTION & THEORY....4
MATERIALS AND EQUIPMENT....5
EXPERIMENTAL SETUP....6
METHODOLOGY & PROGRAMMING LOGIC....7
DATA COLLECTION....8
DATA ANALYSIS....9
CODE....9
RESULTS & DISCUSSION....10
CONCLUSION....10
RECOMMENDATIONS....10
REFERENCES....10
APPENDICES....10
ACKNOWLEDGEMENT....10
STUDENT DECLARATION....10

INTRODUCTION & THEORY

A seven-segment display consists of seven individual segments labelled A to G. Different combinations of these segments are used to display numbers from 0 to 9.

In this experiment, an Arduino Uno was used to control a common-anode seven-segment display. Since the display is common-anode, a LOW signal turns a segment ON, while a HIGH signal turns it OFF.

Two pushbuttons were used to control the display. One button increases the displayed number, while the other resets the counter to 0.

The objective of this experiment was to interface the seven-segment display and pushbuttons with the Arduino and produce a working 0–9 counter.

MATERIALS AND EQUIPMENT

The following components were used:

- Arduino Uno
- Common-anode 7-segment display
- 7 × 220 Ω resistors
- 2 × pushbuttons
- Breadboard
- Jumper wires
- Laptop with Arduino IDE

EXPERIMENTAL SETUP

- The seven-segment display was connected to the Arduino using the following pins:

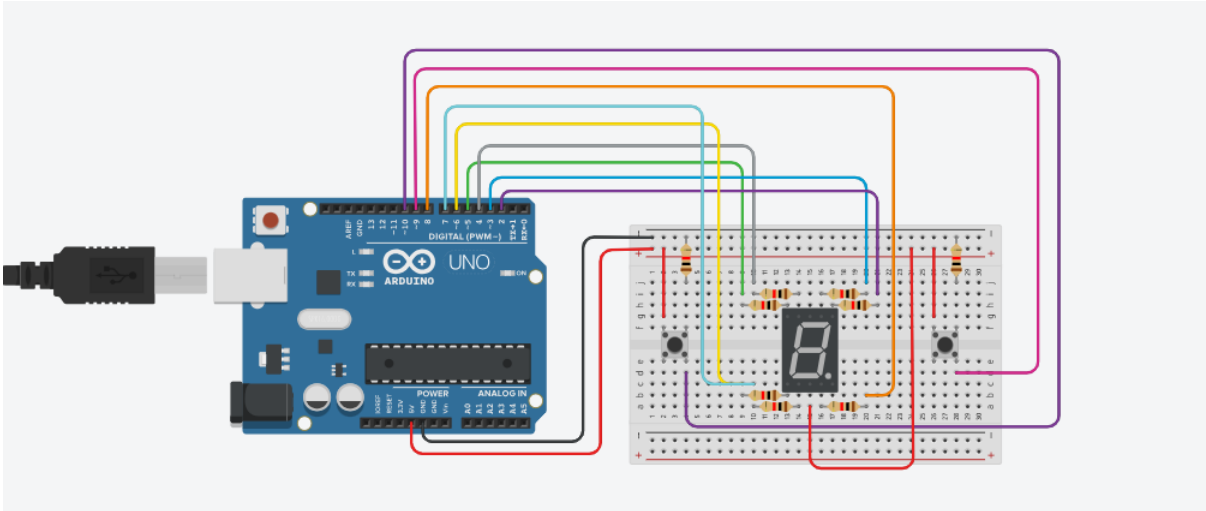
Segment	Arduino Pin
A	D4
B	D5
C	D8
D	D7
E	D6
F	D3
G	D2

- The two pushbuttons were connected as follows:

Function	Arduino Pin
Count Up	D9
Reset	D10

- The display segments were connected through 220 Ω resistors to limit the current.
- The pushbuttons are connected to 5V alongside ground and the inputs are connected opposite of it.
- 10K-ohm pull-up resistors are used for each pushbutton by connecting one end of each resistor to the digital pin and the other end to the 5V output of the of the Arduino.

- Hardware configuration instructions.



METHODOLOGY & PROGRAMMING LOGIC

The experiment was carried out by:

1. Building the circuit according to the required connections.
2. Connecting the seven-segment display and two pushbuttons to the Arduino.
3. Uploading the Arduino program.
4. Starting the counter at 0.
5. Pressing the count-up button to increase the displayed number.
6. Testing the counter from 0 to 9.
7. Pressing the reset button to return the display to 0.

The `void displayDigit(int num) {` function controls which segments are ON or OFF for each number.

The count-up function uses:

```
count = (count + 1) % 10;
```

This makes the counter return to 0 after reaching 9.

The reset function uses:

```
count = 0; // Reset counter to 0
```

A 200 ms delay is included after each button press to reduce button bouncing.

DATA COLLECTION

- The seven-segment display successfully showed all digits from 0 to 9 in sequence.
- The digital output states (HIGH or LOW) act as the data sent to pins D2-D8 to turn each LED segment on or off.
- The Serial Monitor functions (Arduino IDE) as the data acquisition instrument, tracking the digital state shifts caused by the increment and reset actions.

DATA ANALYSIS

A common-anode 7-segment display requires **LOW voltage signals** to illuminate specific segments. By controlling the combination of LOW and HIGH states across the seven digital pins, different numerical patterns are formed. For example, setting segments A, B, C, D, E, and F to LOW while keeping segment G HIGH displays the number **0**.

The pushbuttons successfully changed the internal logic states of the Arduino, allowing the count-up button to increase the displayed number and the reset button to return the display to 0.

CODE

MODULE2-QUEST.ino

```
// Segment pin definitions based on your setup
const int segmentA = 4; // D0
const int segmentB = 5; // D1
const int segmentC = 8; // D2
const int segmentD = 7; // D3
const int segmentE = 6; // D4
const int segmentF = 3; // D5
const int segmentG = 2; // D6

// Pushbutton pins
const int buttonPin = 9;      // Count Up Button
const int resetButtonPin = 10; // Reset Button

int count = 0;
bool lastButtonState = HIGH;
bool lastResetButtonState = HIGH;

// Helper function to render a number (Common Anode: LOW = ON, HIGH = OFF)
void displayDigit(int num) {
  switch (num) {
    case 0:
      digitalWrite(segmentA, LOW);
      digitalWrite(segmentB, LOW);
      digitalWrite(segmentC, LOW);
      digitalWrite(segmentD, LOW);
      digitalWrite(segmentE, LOW);
      digitalWrite(segmentF, LOW);
      digitalWrite(segmentG, HIGH);
      break;
    case 1:
      digitalWrite(segmentA, HIGH);
      digitalWrite(segmentB, LOW);
```

```
      digitalWrite(segmentC, LOW);
      digitalWrite(segmentD, HIGH);
      digitalWrite(segmentE, HIGH);
      digitalWrite(segmentF, HIGH);
      digitalWrite(segmentG, HIGH);
      break;
    case 2:
      digitalWrite(segmentA, LOW);
      digitalWrite(segmentB, LOW);
      digitalWrite(segmentC, HIGH);
      digitalWrite(segmentD, LOW);
      digitalWrite(segmentE, LOW);
      digitalWrite(segmentF, HIGH);
      digitalWrite(segmentG, LOW);
      break;
    case 3:
      digitalWrite(segmentA, LOW);
      digitalWrite(segmentB, LOW);
      digitalWrite(segmentC, LOW);
      digitalWrite(segmentD, LOW);
      digitalWrite(segmentE, HIGH);
      digitalWrite(segmentF, HIGH);
      digitalWrite(segmentG, LOW);
      break;
    case 4:
      digitalWrite(segmentA, HIGH);
      digitalWrite(segmentB, LOW);
      digitalWrite(segmentC, LOW);
      digitalWrite(segmentD, HIGH);
      digitalWrite(segmentE, HIGH);
      digitalWrite(segmentF, LOW);
```

```
      digitalWrite(segmentG, LOW);
      break;
    case 5:
      digitalWrite(segmentA, LOW);
      digitalWrite(segmentB, HIGH);
      digitalWrite(segmentC, LOW);
      digitalWrite(segmentD, LOW);
      digitalWrite(segmentE, HIGH);
      digitalWrite(segmentF, LOW);
      digitalWrite(segmentG, LOW);
      break;
    case 6:
      digitalWrite(segmentA, LOW);
      digitalWrite(segmentB, HIGH);
      digitalWrite(segmentC, LOW);
      digitalWrite(segmentD, LOW);
      digitalWrite(segmentE, LOW);
      digitalWrite(segmentF, LOW);
      digitalWrite(segmentG, LOW);
      break;
    case 7:
      digitalWrite(segmentA, LOW);
      digitalWrite(segmentB, LOW);
      digitalWrite(segmentC, LOW);
      digitalWrite(segmentD, HIGH);
      digitalWrite(segmentE, HIGH);
      digitalWrite(segmentF, HIGH);
      digitalWrite(segmentG, HIGH);
      break;
    case 8:
      digitalWrite(segmentA, LOW);
      digitalWrite(segmentB, LOW);
```

```
      digitalWrite(segmentC, LOW);
      digitalWrite(segmentD, LOW);
      digitalWrite(segmentE, LOW);
      digitalWrite(segmentF, LOW);
      digitalWrite(segmentG, LOW);
      break;
    case 9:
      digitalWrite(segmentA, LOW);
      digitalWrite(segmentB, LOW);
      digitalWrite(segmentC, LOW);
      digitalWrite(segmentD, LOW);
      digitalWrite(segmentE, HIGH);
      digitalWrite(segmentF, LOW);
      digitalWrite(segmentG, LOW);
      break;
  }
}

void setup() {
  pinMode(segmentA, OUTPUT);
  pinMode(segmentB, OUTPUT);
  pinMode(segmentC, OUTPUT);
  pinMode(segmentD, OUTPUT);
  pinMode(segmentE, OUTPUT);
  pinMode(segmentF, OUTPUT);
  pinMode(segmentG, OUTPUT);

  // Configure both buttons with internal pull-ups
  pinMode(buttonPin, INPUT_PULLUP);
  pinMode(resetButtonPin, INPUT_PULLUP);

  displayDigit(count); // Start at 0
}

void loop() {
  bool currentButtonState = digitalRead(buttonPin);
  bool currentResetButtonState = digitalRead(resetButtonPin);

  // Check Count Up Button Press
  if (lastButtonState == HIGH && currentButtonState == LOW) {
    count = (count + 1) % 10;
    displayDigit(count);
    delay(200); // Debounce delay
  }

  // Check Reset Button Press
  if (lastResetButtonState == HIGH && currentResetButtonState == LOW) {
    count = 0; // Reset counter to 0
    displayDigit(count);
    delay(200); // Debounce delay
  }

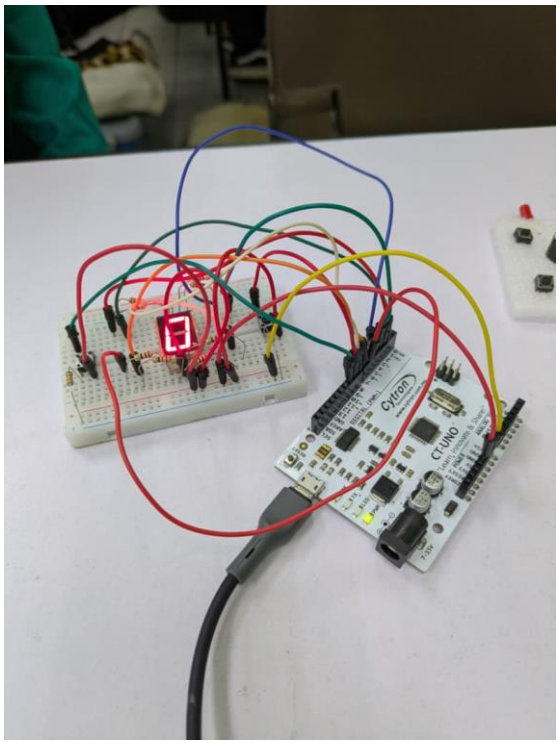
  lastButtonState = currentButtonState;
  lastResetButtonState = currentResetButtonState;
}
```

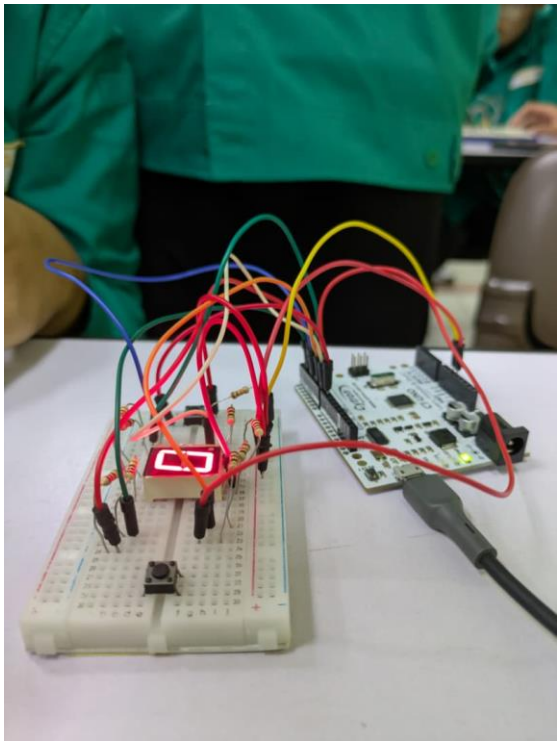
RESULTS & DISCUSSION

The experiment was successfully completed. The seven-segment display displayed the numbers from 0 to 9 correctly.

The count-up button successfully increased the displayed number, while the reset button returned the display to 0. No significant problems were encountered during testing.

The common-anode configuration required LOW signals to activate the display segments. Therefore, the program was written according to the active-LOW operation of the display.





Visual evidence:

https://drive.google.com/drive/folders/1OyrHAU-J05q-vPYV7gfLzClhoJEGqY_4?usp=drive_link

Questions:

How can you interface an I2C LCD with Arduino? Explain the coding principle behind it compared to a 7-segment display and a matrix LED.

Answer:

An I²C LCD can be connected to an Arduino using two communication lines: **SDA** and **SCL**. For an Arduino Uno, SDA is connected to A4 and SCL is connected to A5.

Unlike the seven-segment display, which requires individual digital pins to control each segment, an I²C LCD receives commands and data through the I²C communication protocol.

A matrix LED uses rows and columns to control multiple LEDs and can display different patterns or characters.

CONCLUSION

The experiment successfully demonstrated how an Arduino Uno can be used to control a seven-segment display and pushbuttons.

The display successfully showed numbers from 0 to 9, while the two buttons controlled the counting and reset functions. The experiment also provided an understanding of digital output control and the difference between common-anode and common-cathode displays.

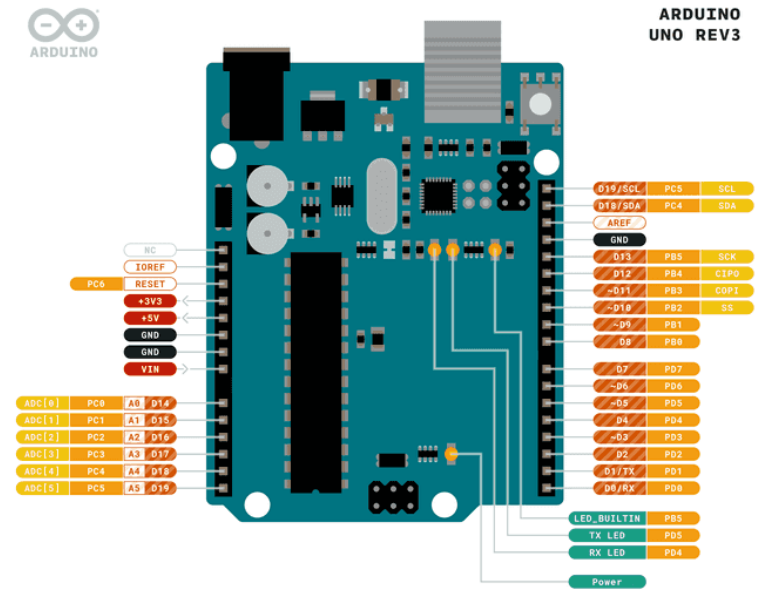
RECOMMENDATIONS

- The system can be improved by adding more seven-segment displays to show larger numbers.
- The program could also be modified to include automatic counting or different counting modes.
- Use arrays to simplify the number of display coding.

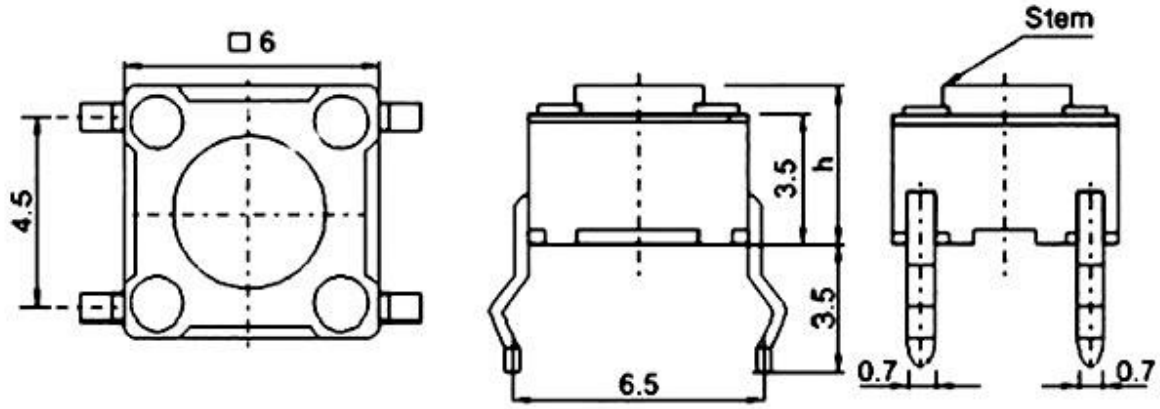
REFERENCES

- Viral Science - The home of Creativity. (2019, February 2). Arduino Seven Segment Display Counter | Push Button | Code Explanation. YouTube. https://www.youtube.com/watch?v=jtmUxgDf6uI
- Fuller, J. (2023, March 4). Button Count on LCD – Arduino Tutorial. Circuits DIY. https://www.circuits-diy.com/button-count-on-lcd-arduino-tutorial/

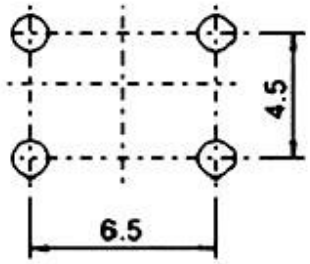
APPENDICES



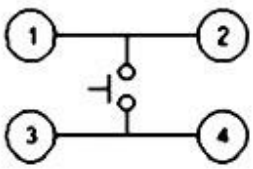




P.C.B.Land Pattern



Circuit Diagram



ACKNOWLEDGEMENT

We would like to express our deepest appreciation to our instructor and lab personnel for their guidance and support throughout this experiment.

STUDENT DECLARATION

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgments, and that the original work contained herein has not been undertaken or done by unauthorized sources or persons.

SIGNATURE	STUDENT DETAILS	CHECKLIST
	Name: Harith Naufal Bin Ahmad Shukri Matric No: 2413381	Read [/] Understand [/] Agree [/]
	Name: Iman Bin Salman Matric No: 2418123	Read [/] Understand [/] Agree [/]
	Name: Luqman Hakimi Bin Suhaizi Matric No: 2411349	Read [/] Understand [/] Agree [/]
	Name: Muhamad Haziq Bin Mohd Shahril Matric No: 2418929	Read [/] Understand [/] Agree [/]